

TEMA 5 – Contenedores

Objetivo del tema

En este tema aprenderás que es posible trabajar dinámicamente con los contenedores de tu interfaz desde tu código fuente Kotlin. Esto puede servirte para añadir cierto dinamismo a las App, pues permite hacer cosas como añadir 'al vuelo' un botón a tu Activity.

5.1 Contenedores

Hasta ahora, hemos estado trabajando con *widgets* y *contenedores* que hemos definido en el `activity_main.xml`. Lo único que teníamos que hacer para trabajar con ellos desde en el `MainActivity.java` era simplemente referenciarlos mediante una variable.

Pero ésta no es la única forma de trabajar con ellos. Es posible recorrer los hijos de un Contenedor en busca de uno en concreto; es posible crear un widget al vuelo en una actividad y asignárselo a un Contenedor; o incluso responder a los eventos de los widget-hijo de un Contenedor. Algo parecido a como trabajas con XML y DOM, dado que sí, tu **activity_main.xml** no deja de ser un árbol xml normal y corriente.

5.2 Recorrer un Contenedor

Si has trabajado anteriormente con xml mediante Java (con librerías), encontrarás similitudes evidentes en Android. En el siguiente ejemplo tienes cómo se recorren los widgets de un contenedor. Es la tarea más básica que se puede hacer. Basta con pasarle la referencia.

```
fun mostrarHijos(container: ViewGroup) {
    for (i in 0 until container.childCount) {
        val hijo = container.getChildAt(i)

        // Mostrar información del hijo
        println("Hijo $i: ${hijo.javaClass.simpleName} - id: ${hijo.id}")
    }
}
```

Hay que tener cuidado porque no todos los widgets de un mismo contenedor son del mismo tipo:

```
fun mostrarSoloBotones(container: ViewGroup) {
    for (i in 0 until container.childCount) {
        val hijo = container.getChildAt(i)

        if (hijo is Button) {
            println("Botón encontrado: id=${hijo.id}, texto='${hijo.text}'")
        }
    }
}
```

5.2 Añadir elementos a un Contenedor

De la misma forma, es posible añadir elementos a un Contenedor que ya existe. Lo único que debes tener en cuenta es que cada nuevo *widget* que añadas necesita una serie de parámetros definidos SI o SI, los llamados **Parámetros de Layout**.

Son los parámetros que se añaden automáticamente en el editor visual cuando añades un *widget*.

Curiosamente, no es lo mismo añadir un widget a un `AbsoluteLayout` que a un `RelativeLayout`. No indicarlos generará errores imprevistos. La forma más fácil de saber cuáles son es usar el método **`setLayoutParams ()`** del widget, pues te pedirá que le pases todos los parámetros que necesita.

```
val contenedor = findViewById<LinearLayout>(R.id.contenedor)
val boton = Button(this)
boton.text = "Soy un botón"
val params = LinearLayout.LayoutParams (
    LinearLayout.LayoutParams.MATCH_PARENT,
    LinearLayout.LayoutParams.WRAP_CONTENT
)
boton.layoutParams = params
contenedor.addView(boton)
```

Recuerda que todos los *widgets* necesitan un ID propio. Para evitar problemas, lo mejor es dejar que Android genere ese ID dinámicamente usando **`generateViewId ()`**.

5.3 Modificación y eliminación de elementos de un Contenedor

Obviamente, si somos capaces de añadir un elemento a un contenedor, somos capaces de modificar los atributos de un widget o incluso eliminarlo dinámicamente.

El problema es que esto genera muchos problemas si no se hace bien... aquí te muestro cómo eliminar un botón de tu layout desde Kotlin.

```
val contenedor = findViewById<LinearLayout>(R.id.miContenedor)
val boton = findViewById<Button>(R.id.miBoton)
contenedor.removeView(boton)
```

Ejercicio 14

Crea una App con un **`GridLayout`** en lugar de un **`ConstraintLayout`**. El `GridLayout` debe de tener tres columnas y seis filas. **`Activity_main.xml`** no debe de tener ningún otro Widget o Layout más que éste. A continuación, añade desde el activity un total de 18 **Botones**.

- Los Button del 1 al 17 no tienen texto, pero cada uno de ellos debe tener un color **aleatorio**.
- El último Button es blanco, y su texto es "Reset".

Cuando se arranque la aplicación, deben de mostrarse estos Buttons. Cuando se pulsa un **Button** cualquiera, ese Button cambia al color a blanco. Cuando se pulsa el "Reset", todos los Button excepto el "Reset" cambian a un color de fondo aleatorio.

Práctica solucionada en **P6 - ColorfullButtons**